# JIT Inventory & Equipment Management System

## System Design & Development Document

**Version 3.1**

Prepared for: Development Team
Last Updated: May 18, 2026

## Table of Contents



## 1. Project Overview

The JIT Inventory & Equipment Management System is a web-based internal tool designed to manage the full lifecycle of company assets — from procurement through disposal — and to handle equipment borrowing by employees through a structured digital workflow.

The system is intended for internal use only. There are no external or public-facing users. All users are company personnel assigned one of three access roles, managed through a fully relational Role-Based Access Control (RBAC) system.

### Core Objectives

- Maintain accurate, real-time inventory records for all company assets
- Track equipment from acquisition through retirement
- Provide a structured digital borrow-and-return process for staff
- Support procurement and supplier management
- Generate reports and audit trails for accountability
- Alert managers to low stock, warranty expirations, and overdue borrows
- Enforce role-based permissions via a database-driven RBAC model from day one

### Scope

The system covers five core operational processes:

- Procurement — Ordering and purchasing of items from suppliers
- Asset Registration — Recording newly acquired items into inventory
- Usage & Monitoring — Tracking stock movement, equipment usage, and condition
- Borrow & Return — Managing temporary equipment borrowing (Workflow B — digital self-request only)
- Disposal & Retirement — Handling old, broken, or unusable equipment

## 2. Technical Stack

| Layer | Technology | Notes |
|---|---|---|
| Frontend | Next.js (App Router) | React framework with SSR and server components |
| Styling | Tailwind CSS + shadcn/ui | Utility-first CSS with pre-built component library |
| Backend | NestJS | Node.js framework with modular architecture |
| ORM | Prisma | Type-safe database client for PostgreSQL |
| Database | PostgreSQL via Supabase | Hosted relational DB — Supabase used as DB host only |
| File Storage | Supabase Storage | Image uploads, warranty documents, digital assets |
| Authentication | Custom NestJS JWT | Two-token strategy (AT in memory + RT in httpOnly cookie) |
| RBAC Guard | Custom NestJS Guard + Prisma | DB-driven permission checks on every request |
| Real-time | Socket.io via NestJS | Live notifications and stock alerts |
| Queue / Jobs | BullMQ + Redis (Upstash) | Scheduled jobs for overdue alerts, low-stock checks |
| Frontend Deploy | Vercel | Next.js native platform, preview deployments per PR |
| Backend Deploy | Railway | Persistent server process; co-hosts Redis add-on |
| State Management | Zustand | Stores access token in memory (never localStorage) |
| HTTP Client | Axios | With response interceptor for token refresh |

### Repository Structure (Monorepo)

```
/
├── apps/
```

```
│   ├── frontend/          # Next.js application
│   └── backend/           # NestJS application
├── packages/
│   └── shared/            # Shared types and constants
├── .github/
│   └── workflows/
│       └── ci.yml         # CI/CD pipeline
└── prisma/
    └── schema.prisma      # Single source of truth for DB schema
```

# 3. Inventory & Equipment Workflow

## Process 1 — Procurement

Purpose: Ensure all inventory and equipment are properly procured and documented before entering the system.

Activities: Record supplier information, create and manage purchase orders with multiple line items, store purchase receipts and invoice attachments, track order status from draft through receipt.

Examples: Buying office materials, purchasing monitors and keyboards, ordering network cables and office supplies.

| Field | Description |
|---|---|
| Purchase Order ID | Unique system identifier |
| Invoice Number | Unique supplier invoice number (duplicates rejected) |
| Supplier | Linked supplier record |
| Order Date | Date the PO was created |
| Status | DRAFT → PENDING → APPROVED → RECEIVED → CANCELLED |
| Total Amount | Computed sum of all line items |
| Receipt / Invoice | Uploaded document attachment |
| Created By | Manager who created the PO |
| Item (line) | Linked inventory item |
| Quantity (line) | Units ordered |
| Unit Cost (line) | Cost per unit |
| Subtotal (line) | Quantity × Unit Cost |

**Business Rules (Procurement):**

- All purchases must be recorded in the system before items can be registered
- One purchase transaction may contain multiple items
- Only Managers and Admins can create or update purchase orders
- Supplier information must be linked to every purchase order
- Duplicate invoice numbers are rejected by the system

## Process 2 — Asset Registration

Purpose: Officially register inventory items and company equipment into the system after procurement.

Activities: Input item details and categorize the asset, assign unique Asset IDs to equipment, record serial numbers and warranty information, upload item images.

Examples: Stocking office supplies, recording 20 HDMI cables, adding printers and routers.
Possible Categories: Equipment, Consumables, Tech Devices, Office Materials.

### Shared Fields (all items)

| Field | Type | Notes |
|---|---|---|
| Item ID | INT (PK) | Auto-incremented |
| Item Name | VARCHAR | Display name |
| Description | TEXT | Detailed description |
| Category | FK → Categories | Must be set before registration |
| Unit | VARCHAR | pcs, reams, boxes, sets, etc. |
| Quantity | INT | Current stock count |
| Reorder Point | INT | Threshold that triggers low-stock alert |
| Status | ENUM | IN_STOCK, LOW_STOCK, OUT_OF_STOCK, ARCHIVED |
| Is Consumable | BOOLEAN | Distinguishes consumable from trackable equipment |
| Barcode / QR | VARCHAR (unique) | For scanning-based lookup |
| Image URL | VARCHAR | Supabase Storage reference |
| Purchase Order | FK → PurchaseOrders | Source procurement record |
| Registered By | FK → Users | |
| Created At | TIMESTAMP | |
| Updated At | TIMESTAMP | |
| Deleted At | TIMESTAMP (nullable) | Soft-delete — NULL means active |

### Equipment-Specific Fields (when Is Consumable = false)

| Field | Type | Notes |
|---|---|---|
| Asset ID | VARCHAR (unique) | System-generated unique asset identifier |
| Serial Number | VARCHAR (unique, nullable) | Manufacturer serial |
| Brand | VARCHAR | Manufacturer name |
| Model | VARCHAR | Model name/number |
| Assignment Type | ENUM | PHYSICAL or DIGITAL |
| Condition | ENUM | NEW, GOOD, FAIR, POOR, DAMAGED |
| Status | ENUM | AVAILABLE, IN_USE, UNDER_MAINTENANCE, DAMAGED, LOST, BORROWED, RETIRED |
| Location | VARCHAR | Storage location or assigned department |
| Acquisition Date | DATE | When acquired |
| Purchase Price | DECIMAL | Unit value (triggers warranty requirement if above ₱5,000) |
| Warranty Start | DATE | Coverage start |

| Field | Type | Notes |
|---|---|---|
| Warranty End | DATE | Coverage end (tracked for expiry alerts) |
| Warranty Provider | VARCHAR | Supplier or third-party |
| Warranty Doc URL | VARCHAR | Supabase Storage reference |
| Replacement Needed | BOOLEAN | Flags for procurement consideration |
| Is Borrowed | BOOLEAN | Live availability state |

**Business Rules (Asset Registration):**

- Every equipment item must have a unique Asset ID — duplicates are rejected
- Duplicate serial numbers are not allowed
- Equipment must be categorized before registration
- Newly registered items default to AVAILABLE status
- Asset records cannot be deleted once assigned to an employee
- Warranty information must be recorded for equipment with a purchase price above ₱5,000
- Equipment images may be attached for verification

## Process 3 — Usage & Monitoring

Purpose: Monitor stock movement, equipment usage, and item condition on an ongoing basis.

Activities: Record stock usage (deductions and additions), monitor available quantity against reorder thresholds, track equipment location and current user, update equipment condition after returns or inspections, record maintenance history.

### Inventory Item Status Values

| Status | Trigger |
|---|---|
| IN_STOCK | Quantity above reorder point |
| LOW_STOCK | Quantity at or below reorder point |
| OUT_OF_STOCK | Quantity reaches zero |
| ARCHIVED | Item manually retired from active inventory |

### Equipment Status Values

| Status | Meaning |
|---|---|
| AVAILABLE | Ready to be borrowed |
| IN_USE | Assigned to an employee |
| UNDER_MAINTENANCE | Maintenance in progress — blocks borrowing |
| DAMAGED | Damage reported — requires incident report |
| LOST | Reported missing |
| BORROWED | Currently checked out via borrow record |
| RETIRED | Disposed of — archived permanently |

### Data Fields (Stock Movement)

| Field | Notes |
|---|---|
| Stock In/Out ID | PK |
| Item | FK → Items |
| Received / Released By | FK → Users |
| Purchase Order | FK → PurchaseOrders (for Stock In from PO) |
| Quantity Added / Removed | Units moved |
| Purpose | Required for all Stock Out transactions |
| Date | Timestamp |
| Notes | Optional remarks |

## Process 4 — Borrow & Return

Purpose: Manage temporary borrowing of company equipment by employees through a structured digital workflow.

Decision: The system uses Workflow B exclusively. All borrow requests must be submitted digitally by the Staff member. Walk-up or verbally processed requests are not supported.

### Workflow B — Digital Self-Request Flow

```
Staff submits request → System records as PENDING
  → Manager reviews pending queue
  → Manager APPROVES → Equipment status: BORROWED
    → Staff returns equipment physically
    → Manager verifies condition → Status: RETURNED
    → Equipment resets to AVAILABLE (or DAMAGED / UNDER_MAINTENANCE)
  → Manager REJECTS (with note) → Status: REJECTED (terminal)
  → Staff cancels before approval → Status: CANCELLED
System flags unreturned past due → Status: OVERDUE
```

### Borrow Status State Machine

```
PENDING ──→ APPROVED ──→ BORROWED ──→ RETURNED
         ↘ REJECTED (terminal)
BORROWED ──→ OVERDUE (system-flagged by scheduled job)
PENDING  ──→ CANCELLED (Staff cancels own before approval)
```

### Data Fields

| Field | Type | Notes |
|---|---|---|
| Borrow ID | INT (PK) | |
| Equipment | FK → Equipment | Only AVAILABLE equipment can be borrowed |
| Borrowed By | FK → Users (Staff) | The requesting employee |
| Approved By | FK → Users (Manager/Admin) | Nullable until acted upon |
| Borrow Date | TIMESTAMP | When approved and equipment handed over |
| Expected Return | DATE | Required at submission |
| Actual Return | TIMESTAMP (nullable) | Set when Manager processes the return |

| Field | Type | Notes |
|---|---|---|
| Return Condition | ENUM (nullable) | Condition assessed at return |
| Status | ENUM | See state machine |
| Notes | TEXT | Optional notes from either party |

## Process 5 — Disposal & Retirement

Purpose: Properly remove inactive, damaged, or obsolete equipment from company operations.

Activities: Submit disposal request with documented reason, Admin reviews and approves disposal, mark equipment as retired and archive the record, flag replacement need where applicable.

### Disposal Reason Values

| Reason | Effect |
|---|---|
| DAMAGED_BEYOND_REPAIR | Equipment automatically set to RETIRED status |
| OUTDATED | Equipment archived; replacement flag set if needed |
| LOST | Equipment marked LOST; incident documentation required |

### Data Fields

| Field | Type | Notes |
|---|---|---|
| Disposal ID | INT (PK) | |
| Equipment | FK → Equipment (unique) | One disposal record per equipment unit |
| Approved By | FK → Users (Admin) | Disposal approval restricted to Admin role |
| Disposal Date | TIMESTAMP | |
| Reason | ENUM | DAMAGED_BEYOND_REPAIR, OUTDATED, LOST |
| Condition | ENUM | Recorded condition at time of disposal |
| Replacement Needed | BOOLEAN | |
| Notes | TEXT | |

# 4. System Behavior

## Functional Requirements (Must Have — MVP)

- Role-based access control — enforced via DB-driven RBAC (Admin, Manager, Staff)
- Track inventory quantity in real time
- Track equipment lifecycle from registration to disposal
- Record full borrowing history
- Monitor and update equipment condition
- Alert low stock levels when quantity reaches reorder point
- Prevent duplicate Asset IDs and serial numbers
- Restrict borrowing when equipment is unavailable or under maintenance

- Record all stock movement with reason tracking
- Soft-delete all records (archive instead of permanent removal)

### Functional Requirements (Should Have — Phase 2)

- Generate inventory, procurement, borrow, and disposal reports
- Export reports to PDF and Excel
- Track purchase history linked to procurement records
- Record equipment maintenance logs
- Track warranty expiration dates
- Notify Managers of expiring warranties
- Allow image upload for items and equipment
- Track updated records (full audit trail)
- Flag equipment that needs replacement
- Block borrowing for Staff with overdue items (automatic restriction)
- Allow Admins to manage roles and permissions through the UI

### Functional Requirements (Nice to Have — Phase 3)

- Generate QR codes and barcodes for assets
- Scan QR/barcodes for faster asset lookup
- Real-time in-system notifications
- Digital Asset Management (licenses, software, access grants)
- Preventive maintenance scheduling

### Non-Functional Requirements

- All data access requires authentication
- All actions are logged in the audit trail
- Deleted records are archived, never permanently removed
- Inventory quantity cannot go below zero (enforced at DB and service layer)
- Unauthorized users cannot access any system route
- The system must handle concurrent users without race conditions on stock quantity
- API response time target: under 500ms for standard CRUD operations
- File uploads restricted by MIME type and size (10MB max per file)
- RBAC permissions are resolved from the database on every request — not from the JWT payload

## 5. Business Rules

### Procurement Rules

- All purchases must be recorded in the system
- One purchase transaction may contain multiple line items
- Only Managers and Admins can record and update purchases
- Supplier information must be linked to every purchase order
- Purchased items cannot be registered into inventory without a procurement record
- Duplicate purchase invoice numbers are not allowed

## Asset Registration Rules

- Every equipment item must have a unique Asset ID
- Duplicate serial numbers are not allowed
- Equipment must be categorized before registration
- Newly registered items default to AVAILABLE status
- Asset records cannot be deleted once assigned to an employee
- Warranty information must be recorded for equipment with a purchase value above ₱5,000
- Equipment images and documents may be attached for verification

## Inventory Management Rules

- Inventory quantity cannot go below zero
- Stock deductions must always include a valid reason
- Consumable inventory automatically reduces after recorded usage
- Low-stock alerts trigger when quantity reaches the item-level reorder point
- All inventory adjustments must be logged in the audit trail
- Archived inventory records cannot be modified

## Usage & Monitoring Rules

- Equipment condition must be updated after every return
- Equipment marked UNDER_MAINTENANCE cannot be borrowed
- Damaged equipment must include a damage report before the record is updated
- Maintenance activities must be recorded in the maintenance log
- Equipment transfer between departments must be logged
- Equipment in use must always have a responsible employee assigned

## Borrow & Return Rules

- All borrow requests must be submitted through the system portal by the requesting employee
- Only AVAILABLE equipment can be borrowed
- Employees cannot borrow equipment already borrowed by or assigned to another employee
- Every borrow request must include an expected return date
- Returned equipment must undergo condition checking by a Manager before the record is closed
- Late returns are automatically flagged OVERDUE by a scheduled system job
- Staff with an active OVERDUE record are blocked from submitting new borrow requests until resolved
- Borrow transactions cannot be deleted once approved — only status-transitioned
- Borrow approvals require authorization from a Manager or Admin
- Staff may cancel their own request only while still in PENDING status

## Employee Accountability Rules

- Employees are responsible for equipment assigned to or borrowed by them
- Employees must return all company assets and be cleared before resignation
- Lost equipment must be reported immediately through the system
- Damaged equipment may require incident documentation
- Accountability history must remain permanently recorded

## Disposal & Retirement Rules

- Only Admins can approve equipment disposal
- Retired equipment cannot be reassigned or borrowed
- Disposal reason must always be recorded
- Disposal records cannot be deleted
- Equipment marked DAMAGED_BEYOND_REPAIR automatically becomes RETIRED and inactive
- Disposal cannot proceed if the equipment currently has an active borrow record
- Retired assets remain archived for historical reporting

## Security & Access Rules

- Users must log in before accessing any part of the system
- System access is role-based — roles and permissions are managed in the database (RBAC)
- Only Admins can modify critical records (user management, disposal approval, system settings)
- All inventory actions must be logged in the audit trail
- Deleted records must be archived instead of permanently removed
- No role can permanently delete borrow records, disposal records, or audit log entries
- Permission assignments to roles can only be modified by Admins
- System-seeded roles (Admin, Manager, Staff) cannot be deleted, only extended

# 6. Authentication & Access Control

## 6.1 Authentication Flow

The system uses a two-token strategy: a short-lived Access Token (AT) kept in JavaScript memory, and a long-lived Refresh Token (RT) stored in an httpOnly cookie the browser manages automatically. This eliminates XSS token theft while still allowing seamless background renewal.

### Login & Token Issuance

```
Staff submits credentials (email + password)
  → NestJS validates via email lookup + bcrypt
  → Invalid → 401 Unauthorized
  → Valid → Issue tokens:
      AT: 15-minute lifetime, stored in Zustand (JS memory only)
      RT: 7-day lifetime, stored in httpOnly Secure SameSite=Strict cookie
  → Send response
```

### Token Refresh (Authenticated Request Flow)

```
Client sends API request with Authorization: Bearer [AT]
  → NestJS JWT Guard verifies signature + expiry
  → Invalid → 401 Unauthorized
    → Axios interceptor fires POST /auth/refresh
    → httpOnly cookie sent automatically by browser
    → On success: new AT stored in Zustand, original request retried
    → If RT also expired/revoked: redirect to /login
  → Valid → RolesGuard resolves permissions from DB
    → Missing permission → 403 Forbidden
    → Authorized → Controller handler returns 200 + data
```

### Refresh Token Storage (Database)

| Field | Purpose |
|---|---|
| token_hash | SHA-256 of the RT — used for verification |
| expires_at | Expiry timestamp |
| revoked_at | Set on logout to invalidate the session |

## 6.2 Role Definitions

Roles are defined as records in the roles table rather than as an enum on the users table. This enables runtime permission management by Admins without requiring schema migrations. The three system-seeded roles are:

| Role | Who They Are | Access Level |
|---|---|---|
| Admin | System administrators | Unrestricted. Full CRUD on every module. Manages users, roles, permissions, system settings, and approves disposals. |
| Manager | Department heads, inventory officers | Operational authority. Manages all inventory, processes borrow requests, handles procurement, suppliers, and maintenance. Cannot manage user accounts or system-level settings. |
| Staff | All developers & staff | End users / requesters. Browse available inventory, submit borrow requests, view own history. Cannot operate inventory, approve requests, or access management features. |

Note: The is_system flag on role records prevents deletion of these three core roles. Admins may create additional roles and assign any combination of permissions to them.

## 6.3 Access Control Matrix

| Module | Admin | Manager | Staff |
|---|---|---|---|
| Dashboard | Full | Full | Read-Only (own data) |
| User Management | Full | — | — |
| Role & Permissions | Full | — | — |
| Category Management | Full | Full | — |
| Inventory & Items | Full | Full | Read-Only |
| Equipment Management | Full | Full | Read-Only |
| Stock Movement (In/Out) | Full | Full | — |
| Purchase Orders | Full | Full | — |
| Supplier Management | Full | Full | — |
| Borrow & Return | Full | Full | Partial |
| Maintenance Logs | Full | Full | — |
| Disposal & Retirement | Full | Partial | — |
| Digital Assets | Full | Full | Read-Only (granted only) |
| Audit Logs | Read-Only | Read-Only | — |
| Reports & Export | Full | Full | — |

| Module | Admin | Manager | Staff |
|---|---|---|---|
| Notification Settings | Full | Partial | — |
| Settings & Configuration | Full | Partial | — |

### Legend

| Symbol | Meaning |
|---|---|
| Full | Create · Read · Update · Delete / Approve |
| Partial | Scoped write access — see module breakdown |
| Read-Only | View only |
| — | No access; route and UI element hidden |

## 6.4 Frontend Route Access

Staff-accessible routes: /dashboard, /inventory, /equipment, /borrow, /digital-assets

Routes hidden entirely from Staff: /stock, /orders/purchase, /suppliers, /maintenance, /disposal, /reports, /settings, /users, /audit-logs, /roles

## 6.5 Security Notes & Risk Mitigations

| Risk | Severity | Mitigation |
|---|---|---|
| XSS steals the access token from memory | Medium | AT lives in JS memory only. 15-min lifetime limits blast radius. Apply strict CSP headers. |
| CSRF abuses the httpOnly RT cookie | Medium | SameSite=Strict on RT cookie. NestJS CSRF guard on /auth/refresh. |
| Brute-force login attempts | High | Rate-limit POST /auth/login via @nestjs/throttler (10 attempts/15 min/IP). Account lockout after 5 consecutive failures. |
| Refresh token not invalidated on logout | High | On logout, set revoked_at = now() on DB record. Refresh endpoint checks this before issuing a new AT. |
| JWT secret exposed | Critical | Store JWT secret in environment variables only — never in source code. Rotate immediately if compromised. |
| Privilege escalation via role tampering | High | Role and permissions are resolved from the DB on each request by RolesGuard, never from the JWT payload. |
| Unauthorized role/permission modification | High | Only Admins can write to roles and role_permissions tables. Enforced at service and guard layer. |

# 7. Database Architecture

## 7.1 Overview

The database uses a relational structure with PostgreSQL (hosted on Supabase, accessed via Prisma ORM). The schema now consists of 18 tables — three new RBAC tables (roles, permissions, role_permissions) replace the former UserRole enum, enabling runtime permission management without migrations.

## 7.2 Entity Relationship Summary

```
── RBAC ──────────────────────────────────────────────────────────────
ROLES ──────────────── has many ──── ROLE_PERMISSIONS
PERMISSIONS ─────────── has many ──── ROLE_PERMISSIONS
USERS ──────────────── belongs to ── ROLES (via role_id)
── Core ──────────────────────────────────────────────────────────────
USERS ──────────────── has many ──── REFRESH_TOKENS
USERS ──────────────── performs ──── INVENTORY_LOGS
USERS ──────────────── receives ──── STOCK_IN
USERS ──────────────── releases ──── STOCK_OUT
USERS ──────────────── borrows via ─ BORROW_RECORDS (as employee)
USERS ──────────────── approves ──── BORROW_RECORDS (as manager)
USERS ──────────────── approves ──── DISPOSALS
USERS ──────────────── creates ───── PURCHASE_ORDERS
USERS ──────────────── records ───── MAINTENANCE_LOGS
CATEGORIES ─────────── contains ──── ITEMS
ITEMS ──────────────── has one ────── EQUIPMENT (when not consumable)
ITEMS ──────────────── has many ────── STOCK_IN / STOCK_OUT / PO_ITEMS
EQUIPMENT ──────────── has many ────── BORROW_RECORDS / MAINTENANCE_LOGS
EQUIPMENT ──────────── has one ─────── DISPOSAL
SUPPLIERS ──────────── supplies ────── PURCHASE_ORDERS
PURCHASE_ORDERS ─────── contains ──── PURCHASE_ORDER_ITEMS
```

## 7.3 Table Definitions

### 1. roles

Stores the system's access roles. Core roles are seeded at initialization and protected by the is_system flag.

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| name | VARCHAR(50) | UNIQUE, NOT NULL | ADMIN, MANAGER, STAFF (extensible) |
| description | TEXT | nullable | |
| is_system | BOOLEAN | NOT NULL, DEFAULT false | Prevents deletion of core roles |
| created_at | TIMESTAMP | NOT NULL, DEFAULT now() | |

### 2. permissions

Stores discrete, named permissions using a resource:action convention (e.g. inventory:create, borrow:approve). All system permissions are seeded on initialization.

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| name | VARCHAR(100) | UNIQUE, NOT NULL | e.g. inventory:create |
| resource | VARCHAR(50) | NOT NULL | e.g. inventory, equipment, borrow |
| action | VARCHAR(50) | NOT NULL | e.g. create, read, update, delete, approve |

| Column | Type | Constraints | Notes |
|---|---|---|---|
| description | TEXT | nullable | Human-readable description |

### 3. role_permissions

Junction table linking roles to permissions. Admins can modify this table at runtime through the Roles & Permissions UI without any schema changes.

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| role_id | INT | FK → roles.id, NOT NULL | |
| permission_id | INT | FK → permissions.id, NOT NULL | |
| — | — | UNIQUE(role_id, permission_id) | No duplicate assignments |

### 4. users

The former UserRole enum column has been replaced by role_id, a FK into the roles table. This allows role assignments to be changed dynamically and new roles to be added without migrations.

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| first_name | VARCHAR(100) | NOT NULL | |
| last_name | VARCHAR(100) | NOT NULL | |
| email | VARCHAR(255) | UNIQUE, NOT NULL | Login identifier |
| password | VARCHAR | NOT NULL | bcrypt hash |
| role_id | INT | FK → roles.id, NOT NULL | ★ RBAC — replaces ENUM |
| is_active | BOOLEAN | NOT NULL, DEFAULT true | |
| created_at | TIMESTAMP | NOT NULL, DEFAULT now() | |
| deleted_at | TIMESTAMP | nullable | Soft-delete |

### 5. refresh_tokens

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| user_id | INT | FK → users.id, CASCADE | |
| token_hash | VARCHAR | UNIQUE, NOT NULL | SHA-256 hash |
| expires_at | TIMESTAMP | NOT NULL | 7-day expiry |
| revoked_at | TIMESTAMP | nullable | Set on logout |

### 6. categories

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| name | VARCHAR(100) | UNIQUE, NOT NULL | |
| type | ItemType ENUM | NOT NULL | EQUIPMENT, CONSUMABLE, etc. |
| description | TEXT | nullable | |

### 7. items

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| item_name | VARCHAR(255) | NOT NULL | |
| category_id | INT | FK → categories.id | |
| unit | VARCHAR(50) | NOT NULL | pcs, reams, boxes, etc. |
| quantity | INT | NOT NULL, DEFAULT 0 | Cannot go below 0 |
| reorder_point | INT | NOT NULL, DEFAULT 0 | Threshold for alert |
| status | ItemStatus | ENUM, NOT NULL, DEFAULT IN_STOCK | |
| is_consumable | BOOLEAN | NOT NULL, DEFAULT false | |

### 8. equipment

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| item_id | INT | FK → items.id, UNIQUE | One extension per item |
| asset_id | VARCHAR(100) | UNIQUE, NOT NULL | System asset tracker |
| serial_number | VARCHAR(100) | UNIQUE, nullable | |
| assignment_type | AssignmentType | ENUM, NOT NULL, DEFAULT PHYSICAL | PHYSICAL or DIGITAL |
| condition | ConditionStatus | ENUM, NOT NULL, DEFAULT NEW | |
| status | EquipmentStatus | ENUM, NOT NULL, DEFAULT AVAILABLE | |
| purchase_price | DECIMAL(10,2) | nullable | ≥ ₱5,000 triggers warranty |

### 9. suppliers

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| supplier_name | VARCHAR(255) | NOT NULL | |
| contact_person | VARCHAR(255) | nullable | |

### 10. purchase_orders

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| supplier_id | INT | FK → suppliers.id | |
| invoice_number | VARCHAR(100) | UNIQUE, nullable | Duplicates rejected |
| status | PurchaseOrderStatus | DEFAULT DRAFT | |
| total_amount | DECIMAL(10,2) | NOT NULL | |

### 11. purchase_order_items

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| purchase_order_id | INT | FK → purchase_orders.id | |
| item_id | INT | FK → items.id | |
| quantity | INT | NOT NULL | |
| unit_cost | DECIMAL(10,2) | NOT NULL | |

### 12. stock_in

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| item_id | INT | FK → items.id | |
| quantity_added | INT | NOT NULL | |
| purchase_order_id | INT | nullable | Linked if from PO |

### 13. stock_out

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| item_id | INT | FK → items.id | |
| quantity_removed | INT | NOT NULL | |
| purpose | TEXT | NOT NULL | Reason required |

### 14. borrow_records

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| equipment_id | INT | FK → equipment.id | |
| borrowed_by_id | INT | FK → users.id | Staff member |
| approved_by_id | INT | FK → users.id, nullable | Manager |
| expected_return | DATE | NOT NULL | Required |

| Column | Type | Constraints | Notes |
|---|---|---|---|
| status | BorrowStatus | ENUM, DEFAULT PENDING | |

### 15. maintenance_logs

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| equipment_id | INT | FK → equipment.id | |
| status | MaintenanceStatus | ENUM, DEFAULT SCHEDULED | Blocks borrowing if in progress |

### 16. disposals

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| equipment_id | INT | FK → equipment.id, UNIQUE | One disposal record per item |
| approved_by_id | INT | FK → users.id | Admin only |
| reason | DisposalReason | ENUM, NOT NULL | |

### 17. inventory_logs

| Column | Type | Constraints | Notes |
|---|---|---|---|
| id | INT | PK, auto-increment | |
| entity_type | VARCHAR(50) | NOT NULL | Immutable audit trail |
| action | LogAction | ENUM, NOT NULL | Full action list mapping |

## 7.4 Enum Definitions (Prisma Schema Extract)

Note: UserRole enum has been removed. Role identity is now resolved from the roles table at runtime.

```
enum ItemStatus         { IN_STOCK; LOW_STOCK; OUT_OF_STOCK; ARCHIVED; }
enum EquipmentStatus    { AVAILABLE; IN_USE; UNDER_MAINTENANCE; DAMAGED; LOST; BORROWED;
RETIRED; }
enum BorrowStatus       { PENDING; APPROVED; REJECTED; BORROWED; RETURNED; OVERDUE; CANCELLED;
}
enum ConditionStatus    { NEW; GOOD; FAIR; POOR; DAMAGED; }
enum AssignmentType     { PHYSICAL; DIGITAL; }
enum DisposalReason     { DAMAGED_BEYOND_REPAIR; OUTDATED; LOST; }
enum MaintenanceStatus  { SCHEDULED; IN_PROGRESS; COMPLETED; CANCELLED; }
enum PurchaseOrderStatus { DRAFT; PENDING; APPROVED; RECEIVED; CANCELLED; }
enum LogAction          { CREATED; UPDATED; DELETED; APPROVED; REJECTED; BORROWED; RETURNED;
DISPOSED; }
```

## 7.5 RBAC Seed Data (Initial State)

The following data is seeded on first migration. Permissions follow the resource:action convention.

### Seeded Roles

| Name | Description | is_system |
|---|---|---|
| ADMIN | Full system access — unrestricted | true |
| MANAGER | Operational authority — inventory and borrow management | true |
| STAFF | End-user access — browse and self-request only | true |

### Seeded Permission Groups (representative)

| Permission Name | Resource | Action | Granted To |
|---|---|---|---|
| inventory:create | inventory | create | Admin, Manager |
| inventory:read | inventory | read | Admin, Manager, Staff |
| inventory:update | inventory | update | Admin, Manager |
| inventory:delete | inventory | delete | Admin |
| equipment:create | equipment | create | Admin, Manager |
| equipment:read | equipment | read | Admin, Manager, Staff |
| borrow:submit | borrow | create | Admin, Manager, Staff |
| borrow:approve | borrow | approve | Admin, Manager |
| disposal:approve | disposal | approve | Admin |
| users:manage | users | create | Admin |
| roles:manage | roles | update | Admin |
| audit_logs:read | audit_logs | read | Admin, Manager |
| reports:export | reports | read | Admin, Manager |

# 8. Cloud Platform & Infrastructure

## 8.1 Supabase Scope

| Supabase Feature | Decision | Reason |
|---|---|---|
| PostgreSQL Database | ✓ Use | Core relational data store via Prisma |
| Supabase Storage | ✓ Use | Image uploads, warranty docs, attachments |
| Supabase Auth | ✗ Not used | Conflicts with NestJS custom JWT + RBAC flow |
| Supabase Realtime | ✗ Not used | Socket.io via NestJS handles connections |

## 8.2 Storage Buckets

| Bucket Name | Allowed MIME Types | Public | Max File Size |
|---|---|---|---|
| item-images | image/jpeg, image/png, image/webp | No (signed URLs) | 10MB |

| Bucket Name | Allowed MIME Types | Public | Max File Size |
|---|---|---|---|
| equipment-images | image/jpeg, image/png, image/webp | No | 10MB |
| warranty-docs | application/pdf | No | 10MB |
| purchase-receipts | application/pdf, image/* | No | 10MB |

### 8.3 Deployment Architecture

| Service | Platform | Role | Est. Monthly Cost |
|---|---|---|---|
| Frontend (Next.js) | Vercel | SSR, static assets | Free – $20 |
| Backend (NestJS) | Railway | Persistent API & WebSockets | ~$5–15 |
| Queue / Cache | Upstash | Serverless Redis for BullMQ | Free – $10 |
| Database & Storage | Supabase | PostgreSQL & File Bucket | Free |

# 9. Module Breakdown

## Phase 1 — MVP (Core System)

- Module 1: Authentication & User Management
- Module 2: Dashboard
- Module 3: Category Management
- Module 4: Inventory & Item Management
- Module 5: Equipment Management
- Module 6: Stock Movement
- Module 7: Role & Permission Management

## Phase 2 — Operational Expansion

- Module 8: Supplier Management
- Module 9: Procurement Management
- Module 10: Borrow & Return Management (Workflow B only)
- Module 11: Notification & Alert Module
- Module 12: Reports & Export
- Module 13: Disposal & Retirement

## Phase 3 — Enhancement & Governance

- Module 14: Maintenance Management
- Module 15: Warranty Management
- Module 16: Inventory Logs & Audit Trail
- Module 17: Asset Assignment
- Module 18: Digital Asset Management
- Module 19: Settings & Configuration
-

# 10. Delivery Timeline

| Iteration | Weeks | Deliverables |
|---|---|---|
| Iteration 1 | Weeks 1–2 | Project Foundation, Repository Setup, Authentication, RBAC DB Initialization & Seed |
| Iteration 2 | Weeks 3–4 | First Functional Features — Equipment CRUD, Inventory handling, Role & Permission UI, Shared layout |
| Iteration 3 | Weeks 5–6 | Operational expansion — Borrow/Return flows, PDF/Excel generation, Permissions testing |
| Iteration 4 | Weeks 7–8 | Performance optimization, deployment configurations, bug fixes, final hand-off |

# 11. WSJF Prioritization

Prioritization follows the Scaled Agile Framework formula:

**WSJF = (Business Value + Time Criticality + Risk Reduction) / Job Size**

Role & Permission Management is elevated to Phase 1 / MVP because the RBAC database schema underpins all other modules. Deferring it would require a breaking migration later.

# 12. Decision Log

| # | Decision | Chosen Option | Rationale |
|---|---|---|---|
| 1 | Role System | 3 seeded roles in DB (Admin, Manager, Staff) | Matches internal structural workflows. DB-managed for RBAC extensibility. |
| 2 | Borrow Workflow | Workflow B only (Digital self-request) | Enforces strict data integrity and a clear digital audit trail. |
| 3 | Backend Framework | NestJS | Modular architecture with enterprise-grade TypeScript & decorator routing. |
| 4 | Frontend Framework | Next.js (App Router) | Native SSR and optimized server component streaming. |
| 5 | Database | PostgreSQL via Supabase | Strong relational enforcement for asset logs and transaction records. |
| 6 | ORM | Prisma | Type-safe client generation matching PostgreSQL schemas. |
| 7 | Authentication | Custom NestJS JWT Two-Token | Secure enterprise cookie storage. Supabase Auth omitted. |
| 8 | High-Value Threshold | ₱5,000 purchase price | Triggers strict warranty document requirement for fiscal accountability. |
| 9 | Overdue Action | Lock request capability | Prevents unreturned asset accumulation across employee rosters. |
| 10 | Role Storage | roles table instead of UserRole ENUM | Enables runtime permission management by Admins without DB migrations. |
| 11 | Permission Model | role_permissions junction table | True RBAC: many roles → many permissions; extendable without code changes. |

| # | Decision | Chosen Option | Rationale |
|---|---|---|---|
| 12 | Permission Resolution | DB lookup on every request (not JWT payload) | Immediate effect on role changes; prevents privilege escalation attacks. |
| 13 | RBAC Timing | Phase 1 / MVP | RBAC schema is a foundation — deferring it would require breaking migrations. |

# 13. Open Items

| ID | Item Description | Priority | Owner |
|---|---|---|---|
| 1 | ERD diagram compilation for all 18 tables including RBAC tables | High | Lead Developer |
| 2 | Initialize Prisma development migrations (including RBAC seed) | High | Backend Team |
| 3 | Configure domain structures for cross-subdomain cookies | High | DevOps / Lead Dev |
| 4 | Seed roles, permissions, and role_permissions on first migration | High | Backend Team |
| 5 | Build Role & Permission management UI (Admin only) | High | Frontend Team |
| 6 | Configure storage buckets with exact size limits on Supabase | Medium | Backend Team |
| 7 | Hook Upstash Redis connection string directly to Railway instances | Medium | Backend Team |
| 8 | Implement RolesGuard to resolve permissions from DB on every request | High | Backend Team |
| 9 | Validate automated overdue cron validation triggers | Low | QA Analyst |